《CUBE RUNNER》开发日志 · 第 1 周

入职第一天：把机器人画到屏幕上

计算机图形学实验（一） | 4 学时

渲染管线 · VAO/VBO · 着色器 · 齐次坐标 · 几何变换 · 层级建模

剧情开场：欢迎入职星穹互娱

你是 **明宇**，C++ 实习生，今天报到。

图形组组长**老周**（保温杯泡枸杞的那位）宣布：

> “公司要参加 Game Jam，作品叫《CUBE RUNNER》。
> 项目组成员：**你，以及你**。
> 美术的建模软件许可证还在采购流程里——
> 所以主角机器人，**用立方体拼，用数学造**。”

四周冲刺计划（实习任务工单）

周	实验	工单	交付物
1	**几何变换与层级建模**	**SD-101 / SD-102**	**几何变换 + 主角机器人** ← 今天
2	场景建模与光照纹理	SD-201 / SD-202	方块镇场景 + Phong 光照
3	观察变换与相机系统	SD-301 ~ 303	WASD 行走 + 第三人称相机
4	角色动画与碰撞检测	SD-401 ~ 403	跑跳动作 + 碰撞 + 交付

四次课 = 一个完整的 OpenGL 游戏 Demo

每周的代码都建立在上一周之上——别弄丢你的工程！

提交与演示要求（重要）

截止日期：第 12 周 · 周五 下午 5:00

机房提交到实验平台

提交要求

实验 1 ~ 实验 4 的**综合实验报告**（根据模板撰写，**PDF** 格式）

演示要求

最后一次课，各班**抽查三分之一**同学演示，需要**精讲代码（讲解不通过者，对应功能不得分）**

抽到但未在现场的同学：需**单独联系老师**，单独演示

若同学有创新，可自愿展示（**加分不超过 10 分**）

我们的技术栈（2026 年正规军配置）

部件	选择	备注
语言	**C++20**	实习生也要用现代 C++
图形 API	**OpenGL 4.1 Core Profile**	可编程管线，全平台通用
窗口	GLFW	事实标准
GL 加载	自研迷你加载器 glcore	看穿 glad/glew 的魔法
数学库	**自研 mgl（明宇图形库）**	← 你的战场

> 老周：“不许用 GLM。矩阵自己推一遍才算真的会。”

一个顶点的一生：渲染管线

顶点数据（VBO） ・ 模型空间坐标

↓

顶点着色器：模型 → 世界 → 相机 → 裁剪空间 **【可编程】**
uModel ・ uView ・ uProj 三次矩阵乘法

↓

裁剪 → 透视除法 → 视口变换 【固定硬件】

↓

光栅化：三角形 → 一堆片段（候选像素） 【固定硬件】

↓

片段着色器：每个片段算一次颜色 **【可编程】**

↓

深度测试 → 帧缓冲 → 屏幕 【固定硬件】

到底哪些能编程？

可编程：顶点着色器、片段着色器
（几何/细分着色器也存在，本课程用不到）

固定但可拨开关：光栅化、裁剪、深度测试
只能 glEnable / glPolygonMode 配置，不能改写算法

记忆法：名字带“**着色器**”的才能写代码；

其余都是硬件流水线，只会按固定配方干活。

固定管线 vs 可编程管线

上一页“顶点的一生”里，两个方括号阶段并非天生可编程——它们曾经是**焊死在显卡里的电路**：

阶段	固定管线（老黄历，焊死）	可编程管线（现在）
顶点变换	glRotatef / glTranslatef，公式固定	**顶点着色器**：MVP 矩阵自己乘
光照颜色	固定 Phong，glLightfv 只能调参数	**片段着色器**：光照模型自己写
特效（雾、阴影…）	有没有全看显卡厂商心情	会写 GLSL （OpenGL 着色器语言）就有
你的角色	调参侠	**写代码的人**

> 老周：“固定管线早在 2008 年就被判了死刑（OpenGL 3.0 宣布废弃）；
> 你们用的 4.1 Core Profile 里它已被**彻底删除**——所以别找了，只能自己写。”

数据搬运：蚂蚁搬家 vs 一次搬满

过去：立即模式（蚂蚁搬家）

```
glBegin(GL_TRIANGLES);
glVertex3f( 0.0f,  0.5f, 0.0f); // 传一个点
glVertex3f(-0.5f, -0.5f, 0.0f); // 再传一个点
glVertex3f( 0.5f, -0.5f, 0.0f); // 又传一个点
glEnd();
```

- 每画一个三角形，CPU 都要把顶点**挤过主板总线**（带宽极窄）发给 GPU
- 画下一个还得重发：无数次低效 CPU→GPU 拷贝
- GPU 大半时间**等米下锅**，并发算力白白闲置

现在：显存常驻（一次搬满，原地开火）

```
// 初始化：几万个顶点一次搬进 VRAM
glBufferData(GL_ARRAY_BUFFER, ...);

// 每帧：只发指令，不搬数据
glBindVertexArray(vao);
glDrawElements(GL_TRIANGLES, 36, ...);
```

- 加载场景时，顶点/索引**一次性打包**进 GPU 常驻显存（VRAM）
- 运行时 CPU 只需绑 VAO + 一条 Draw Call，**零数据拷贝**
- GPU 在自家显存取数，数百条管线并发计算

> 同一份数据，搬运方式不同 → **性能相差成百上千倍**——这就是 VBO / EBO 存在的意义。

图形学第一公式

```
gl_Position = uProj * uView * uModel * vec4(aPos, 1.0);
```

矩阵	作用	谁写
uModel	模型空间 → 世界空间（摆放）	**本周的你**
uView	世界空间 → 相机空间（观察）	第 3 周的你
uProj	相机空间 → 裁剪空间（透视）	第 3 周的你

前两周 uView/uProj 由“引擎组”临时支援

——第 3 周他们会**收回支援**（先埋个伏笔）

数据怎么上 GPU：VAO / VBO / EBO

```
glGenBuffers(1, &vbo);              // VBO：顶点数据搬进显存
glBufferData(GL_ARRAY_BUFFER, ..., GL_STATIC_DRAW);

glGenBuffers(1, &ebo);              // EBO：索引（哪3个顶点拼1个三角形）

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
                      stride, (void*)0);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, stride,
                      (void *)(3 * sizeof(float)));
 glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, stride,
                      (void *)(6 * sizeof(float)));  // 内存布局说明书
// 布局：|位置xyz|法线xyz|纹理uv| = 8 个 float 一个顶点
```

```
// 着色器GLSL
static const char* kVertexSrc = R"(#version 410 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aNormal;
layout(location = 2) in vec2 aUV;
```

VAO 把上面这套配置录成“状态快照”，画的时候一句 `glBindVertexArray` 即可

完整注释版在 `common/framework/mesh.cpp`——**回去必读**

为什么是 4×4 矩阵？齐次坐标

3×3 矩阵能旋转、能缩放，但**做不了平移**（平移不是线性变换）

数学家的骚操作：给点加个 w=1 → (x, y, z, 1)

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix}$$

平移、旋转、缩放从此统一为矩阵乘法 → **可以随意串联**

这是整个 3D 游戏工业的地基

三个基本变换：平移与缩放

平移 T(tx, ty, tz)：第 4 列填偏移

缩放 S(sx, sy, sz)：对角线填倍率

```
|1 0 0 tx|     |sx 0  0  0|
|0 1 0 ty|     |0  sy 0  0|
|0 0 1 tz|     |0  0  sz 0|
|0 0 0 1 |     |0  0  0  1|
```

mgl.h 里用 r.at(行, 列) = 值 填数——先行后列，和数学书一致

三个基本变换：旋转（要背的三张脸）

绕X轴 Rx(θ)：

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

绕Y轴 Ry(θ)：

$$\begin{bmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

绕Z轴 Rz(θ)：

$$\begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 角度为正 = **右手法则**（拇指指向轴正向，四指是旋转方向）
- 注意 **Ry 的 -s 位置与众不同**——坐标轴轮换 X→Y→Z→X 的结果
- 课堂快问：Ry(90°) 会把 +Z 轴转到哪里？

图形学第一坑：乘法顺序

矩阵乘法**不满足交换律**。对列向量 v' = M · v，变换**从右往左**生效：

```
mat4 M = translate(5,0,0) * rotateY(PI/2) * scale(2,2,2);
//           ③ 最后平移       ② 再旋转       ① 先缩放
```

口诀：先自转，再公转

写反的下场（今天下午你大概率会亲眼看到）：

想让手臂绕肩膀转 → 结果手臂绕自己肚脐转成**螺旋桨**

层级建模：主角“小宇”的身体结构

7 个立方体 + 一套矩阵族谱：

```
root = T(世界位置) · Ry(朝向)
  ├── 躯干 = root · T(0, 0.90, 0) · S(0.50, 0.60, 0.28)
  ├── 头   = root · T(0, 1.26, 0) · Ry(headYaw) · T(0,0.16,0) · S(...)
  │     └── 眼罩 = head · T(0, 0.20, 0.165) · S(...)  ← 孙子级！
  ├── 左臂 = root · T(肩) · Rz(侧抬) · Rx(前摆) · T(0,-0.30,0) · S(...)
  └── 左腿 = root · T(髋) · Rx(前摆) · T(0,-0.31,0) · S(...)
```

改 root → 全身跟着动；改肩关节角 → 只有手臂动

第 4 周的走路/跳跃动画，全靠今天这套族谱

四肢的三明治结构（本周精髓）

$$M_{arm} = root \cdot T(\text{关节点}) \cdot R(\text{关节角}) \cdot T(\text{杆件偏移}) \cdot S(\text{尺寸})$$

从右往左读：

1. **S**：把单位立方体压成“手臂杆”
2. **T(偏移)**：把杆的**顶端**（不是中心）挪到原点 → 原点=肩膀
3. **R**：绕肩膀转（关节角）
4. **T(关节点)**：整条手臂挂到躯干肩部

顺序错一步，手臂就变螺旋桨——**旋转永远绕当前原点**

今天的惨状（运行 lab1_starter）

启动后你会看到：**一坨方块**

原因：`mgl.h` 的 5 个变换函数全部返回**单位矩阵**（“什么都不做”）
所有部件挤在原点，机器人处于“被液压机处理过”的状态

> 老周：“每写对一个函数，屏幕上就有一次肉眼可见的变化。
> 这叫**即时反馈驱动开发**。”

写对	现象
translate	部件散开各就各位
scale	获得正确身材比例
rotateY	随展示台旋转起来
rotateX/Z	摆姿势（按 2/3/4）

动手任务（工单发布）

工单 SD-101（starter/src/mgl.h）：

实现 translate scale rotateX rotateY rotateZ

矩阵模板都印在 TODO 注释上方，**写一个→编译→看效果**

工单 SD-102（starter/src/robot.h）：

躯干和头是示范代码，**左臂/右臂/左腿/右腿归你**

用姿势键做单元测试：2 挥手 | 3 T-pose | 4 行走定格

自由发挥：加天线、背包、帽子……记入实验报告加分

验收清单（老周就位）

1. ☐ 机器人 7 部件位置比例正确，随展示台平滑旋转
2. ☐ 按 2/3/4 时四肢绕**关节点**旋转（不是绕部件中心）
3. ☐ 说清楚：为什么平移需要齐次坐标？
4. ☐ 说清楚 + 演示：`T·R·S` 写成 `R·T·S` 会发生什么？

> 老周：“3、4 才是重点。代码网上到处有，
> **能讲明白的脑子只能自己长**。”

加班挑战（选做）：锥形帽 ☆ | 两段式手臂 ☆☆ | 任意轴旋转 ☆☆☆ | 四元数 ☆☆☆☆

下周预告

主角有了，但他站在一片虚无里
像个赛博孤儿

下周：策划入职，关卡设计稿到货
搭出方块镇 + 让世界有光

（顺便回答一个问题：
为什么你的机器人看起来像块塑料？）

第 2 周：无中生有——场景建模与 Phong 光照